

Proposition de sujet pratique contextualisé, calibré pour une session de 3 heures à effectuer en trinôme (ou binôme).

Sujet est conçu pour un public en formation continue :

Le scénario simule une problématique professionnelle concrète (analyse de données de ventes et gestion de stock d'une PME) afin de mobiliser directement les compétences fondamentales de Python.

Examen Pratique Python :

Système de Gestion et d'Analyse des Ventes (*LogiTech Distribution*)

- Durée : 3 heures
- Format : Travail en groupe (3 personnes idéalement)
- Livrables attendus : Un script Python unique .py ou un notebook .ipynb commenté et exécutable.

Contextualisation du projet

Vous faites partie de l'équipe informatique de la société LogiTech Distribution, un grossiste en matériel bureautique. La direction souhaite remplacer la saisie manuelle de fin de journée par un petit script Python capable de :

- Structurer les données de ventes reçues.
- Appliquer les règles de calcul de taxes et remises applicables.
- Alerter sur le niveau de stock critique.
- Générer un rapport synthétique pour la direction.

Compétences mise en œuvres :

- Manipulation des variables et types fondamentaux (int, float, str, bool).
- Structures de données (listes, dictionnaires, tuples).
- Structures de contrôle (if / elif / else, boucles for / while).
- Fonctions (définition, arguments, valeurs de retour).
- Bonnes pratiques de code (commentaires, modularité).

Les temps de réalisation des différentes parties ne sont que des propositions horaire.

Partie 1 : Chargement et structuration des données (45 min) :

L'objectif de cette étape est de définir les données d'entrée de l'entreprise sous forme de structures Python appropriées.

- ❖ Stock initial : Créez un dictionnaire stock dont chaque clé est un identifiant de produit (ex: "P01") et la valeur est un dictionnaire contenant :

1. nom (ex: "Ecran 24 pouces")
2. prix_unitaire_ht (float)
3. quantite_dispo (int)
4. seuil_alerte (int)

Entrez au moins 5 produits différents.

- ❖ Registre des ventes du jour : Créez une liste de tuples ventes_jour représentant les transactions sous la forme (id_produit, quantite_vendue).

Partie 2 : Développement du moteur de règles et fonctions (1h00) :Pensez à découper votre code en fonctions réutilisables.

Développez les fonctions logiques indispensables aux calculs métier :

- ❖ Fonction calculer_montant_ttc(prix_ht, quantite) :

1. Calcule le montant HT total.
2. Applique la TVA standard de 20%.
3. Applique une remise commerciale de 10% si le montant HT total dépasse 500 €.
4. Retourne le montant TTC final arrondi à 2 décimales.

❖ Fonction `traiter_transaction(id_produit, quantite)` :

1. Vérifie si l'identifiant existe dans le stock.
2. Vérifie si la quantité demandée est disponible en stock.
3. Si le stock est suffisant : décrémenter la quantité du stock et renvoie le montant TTC calculé.
4. Sinon : affiche un message d'erreur clair et annule la transaction.

Partie 3 : Traitement global et rapport d'alerte (1h00) :

Créer le script principal qui va consommer la liste `ventes_jour` et produire un rapport final dans la console.

- ❖ Bouclez sur l'ensemble de la liste `ventes_jour` et traitez chaque vente.
- ❖ Calculez :

1. Le chiffre d'affaires total du jour (TTC).
2. Le nombre total d'articles vendus.
3. Le nombre de transactions échouées (faute de stock ou produit invalide).

- ❖ Rapport de stock critique : À la fin du traitement, votre script doit parcourir le stock mis à jour et afficher la liste de tous les produits dont le niveau restant est inférieur ou égal au seuil_alerte.

Partie 4 : Interface utilisateur en ligne de commande - Optionnel (15 min) : Bonus pour les groupes ayant terminé en avance.

Proposez un menu textuel interactif avec une boucle `while` permettant à un utilisateur d'ajouter manuellement une nouvelle vente, d'afficher l'état des stocks ou de quitter le programme.

Pour information :

Critère d'évaluation	Description
Structure des données	Choix cohérent et bonne manipulation des listes et dictionnaires
Logique et conditions	Respect des règles de calcul (remises, calcul de TVA, vérifications de stock)
Modularité (Fonctions)	Bon découpage du code, paramètres pertinents et valeurs de retour
Traitement des boucles	Parcours efficace des listes/dictionnaires et gestion des alertes
Qualité du code & Travail d'équipe	Clarté du code, nommage des variables, présence de commentaires, lisibilité des affichages